# V3 ESP32 WiFi CSI Occupancy Detection

Complete project handover for a new collaborator

## Purpose

V3 is a research pipeline for detecting room presence and motion state from ESP32 WiFi Channel State Information under room, hardware, and measured WiFi-channel changes. This PDF summarizes the implementation, approved model files, saved evidence, reproduction commands, and remaining work.

## Executive summary

• Presence task: empty versus occupied.

• Motion task: empty, stationary person, or moving person.

• Common deep input: float16 relative-amplitude sequence [32, 60] plus a bool[60] packet-gap mask.

• Sampling is irregular at acquisition; 20 Hz is a preprocessing grid, not a native ESP32 rate.

• The strongest V3 evidence is real room transfer, hardware variation, and a measured congested-channel test.

• A 30-second empty-room calibration improves the N3 held-out-room presence result from about 0.47 to 0.72 balanced accuracy and reduces false alarms from 100% to about 10% for the reported ensemble.

• The final deployment files were trained after the historical held-out runs and are not the exact evaluation checkpoints.

# 1. End-to-end system

A transmitter ESP32 sends regular WiFi packets. A receiver ESP32 measures CSI for packets it receives and streams records over USB serial. The host parses the records, converts I/Q to amplitude, filters unstable bins, resamples the irregular stream to 20 Hz, creates 3-second windows every second, and runs LightGBM, CNN, or transformer inference. Calibration and five-window smoothing are separate runtime/evaluation operations.

## Data flow

| Stage | Implementation | Output |
|---|---|---|
| Acquisition | Two ESP32 boards; receiver USB serial | CSI_DATA records |
| Parsing | esp32_io.py: parse_raw_lines, lltf_amplitude | timestamp, RSSI, L-LTF amplitude |
| Resampling | preprocess.py: to_uniform_grid | 20 Hz signal plus missing mask |
| Windowing | build.py: stream_windows | 3 s / 1 s-hop windows |
| Representation | features.py: relative_fluctuation, cnn_input | 32 bins x 60 samples |
| Prediction | deep.py or LightGBM | presence or motion logits |
| Postprocessing | calibrate_v3.py and experiments_noise.py | calibrated/smoothed label |

The receiver parser supports 104, 128, or 384 interleaved I/Q values and retains the L-LTF portion. The firmware documentation also contains an amplitude-only capture format; that format is not sufficient for the complete V3 timestamp/RSSI/IQ contract without an adapter.

## Hardware requirements

- Two ESP32 DevKit boards, two USB cables, and a laptop or Raspberry Pi host.
- ESP-IDF for flashing; receiver/transmitter firmware under datasets/esp32_csi_pipeline/firmware/.
- Receiver serial at 460800 baud; capture dependency pyserial>=3.5.
- Keep board positions, channel, bandwidth, and link arrangement fixed during calibration and inference.

# 2. Preprocessing and features

For each packet, the parser retains timestamp, RSSI, CSI length, metadata, and raw interleaved I/Q. Amplitude is computed as |H| = sqrt(I^2 + Q^2). valid_subcarriers removes bins whose median amplitude is below 0.15 or above 4.0 times the typical level. relative_fluctuation divides each subcarrier by its window median, subtracts one, and clips robust outliers at six MADs.

The irregular packet stream is mapped to floor(time*20) bins. Packets sharing a bin are averaged. Empty bins are linearly interpolated, but missing=True is retained. build.py then creates 60-sample windows every 20 samples and resamples surviving subcarriers to 32 bins. This preserves a diagnostic gap mask but loses native packet timing detail beyond the 20 Hz grid.

## Feature families implemented for LightGBM

| Group | Implemented features | Current status |
|---|---|---|
| Motion | std, robust std/MAD, first-difference energy, range, quantiles | Current engineered baseline |
| PCA/shape | top eigen-energy, effective rank, participation, correlations | Implemented; not separately ablated |
| Spectrum | band powers, centroid, entropy, autocorrelation | Implemented; needs grouped ablation |
| Periodicity | peak share, flatness, periodic ACF, peak frequency | Implemented; hypothesis for fans/AC |
| RSSI | standard deviation, range, absolute differences | Implemented dynamics only; no absolute RSSI |
| Sequence | relative amplitude [32,60] | CNN and transformer input |
| Phase/IQ | raw archive retained where available | Not used by V3 models |

The transformer consumes the [60] gap mask as learned padding tokens. CNN1D.forward accepts gap but ignores it. LightGBM receives engineered features only.

# 3. Approved V3 model files

Use only these six classifier artifacts for V3 inference. They are state dictionaries/LightGBM boosters and require the matching definitions in src/deep.py or LightGBM.

| Task | Family | File | Classes | Load |
|---|---|---|---|---|
| Presence | LightGBM | models/presence_lgbm.txt | empty, occupied | lgb.Booster(model_file=...) |
| Presence | CNN | models/presence_cnn.pt | empty, occupied | deep.make_model("cnn", n_classes=2) |
| Presence | Transformer | models/presence_tf.pt | empty, occupied | deep.make_model("tf", n_classes=2) |
| Motion | LightGBM | models/motion_lgbm.txt | empty, static, moving | lgb.Booster(model_file=...) |
| Motion | CNN | models/motion_cnn.pt | empty, static, moving | deep.make_model("cnn", n_classes=3) |
| Motion | Transformer | models/motion_tf.pt | empty, static, moving | deep.make_model("tf", n_classes=3) |

V3 also contains models/transformer_pretrained.pt. It is initialization-only, has no occupancy/motion head, and must not be used as a standalone runtime model. V2/models and V4/models are separate-version artifacts.

The registry load-tested 7/7 V3 .pt/support artifacts. Exact sizes and SHA-256 values are in manifests/models.json.

## Model-to-result mapping

| Runtime file | Saved JSON variant | Task/result scope |
|---|---|---|
| presence_lgbm.txt | lgbm | Presence N1-N4 and calibrated counterparts where available |
| presence_cnn.pt | cnn | Presence N1-N4 and calibrated counterparts where available |
| presence_tf.pt | tf | Presence N1-N4 and calibrated counterparts where available |
| motion_lgbm.txt | lgbm | Motion N1-N3 and calibrated counterparts where available |
| motion_cnn.pt | cnn | Motion N1-N3 and calibrated counterparts where available |
| motion_tf.pt | tf | Motion N1-N3 and calibrated counterparts where available |

Ensemble rows are probability averages from evaluation and do not correspond to a separate checkpoint. Calibration changes inputs at runtime/evaluation; it does not create separate weights.

# 4. Datasets and protocols

| Dataset | Contribution | Current use |
|---|---|---|
| ESPectre | Real homes, rooms, multiple ESP32 chip types | N1/N1c and final V3 training |
| esp32-csi-pipeline | Two rooms, quiet and congested channel recordings | N2/N2c/N3/N3c |
| WiCount | ESP32-S3 labelled people-count CSV | Reference/N4 source |
| WiGesture/WiFall | ESP32 activity recordings | Transformer masked pretraining |
| OTH thesis | ESP-NOW HT40 labelled reference | N4 source/reference |
| OpenCSI | Bursty mesh-link recordings | Excluded from V3 scores |
| HomeOccupancy | ESP32-C6 home empty/sleep/work | Separate V4 fine-tuning |

## Protocols

• N1: leave one ESPectre room out; train on the other rooms.

• N2: train on quiet pipeline recording; test same room on measured congested channel.

• N3: train pipeline room 1; test unseen room 2.

• N4: train on other ESP32 data; test pipeline setup.

• N1c/N2c/N3c: corresponding protocols with a 30-second empty-room reference; reference windows are excluded from scoring.

The calibration site key is dataset + room + chip + channel. Energy features are log-ratio calibrated and each sequence bin is divided by an empty-reference robust level. Five-window median smoothing is applied within recordings for smoothed scores.

## Historical source files

N1_espectre_leave_room_out.json, N1c_espectre_leave_room_out_calibrated.json, N2_pipeline_quiet_to_congested.json, N2c_pipeline_quiet_to_congested_calibrated.json, N3_pipeline_room1_to_room2.json, N3c_pipeline_room1_to_room2_calibrated.json, and N4_all_to_pipeline.json are under results/. The canonical flattened table is results/metrics/v3_metrics.csv.

# 5. Verified results

The table below is computed from the saved canonical CSV. It summarizes smoothed presence rows for the principal protocols; the full CSV retains every model, task, room, window/smoothed value, and confusion matrix.

| Protocol | Model | Balanced accuracy | False alarm | Occupied sensitivity |
|---|---|---|---|---|
| N2 raw | Transformer | 0.937 | 0.016 | 0.984 |
| N2 calibrated | Ensemble | 0.910 | 0.028 | 0.992 |
| N3 raw | Ensemble | 0.472 | 1.000 | 1.000 |
| N3 calibrated | Ensemble | 0.719 | 0.095 | 0.991 |
| N4 raw | Transformer | 0.674 | 0.647 | 0.994 |

These headline values are source-result values, not estimates from prose. N3 demonstrates the central V3 problem: an unseen room causes near-universal empty-room false alarms without calibration, while a short empty-room reference substantially improves the ensemble. Calibration is helpful but not universally sufficient; N1 varies by held-out home room.

## Figures

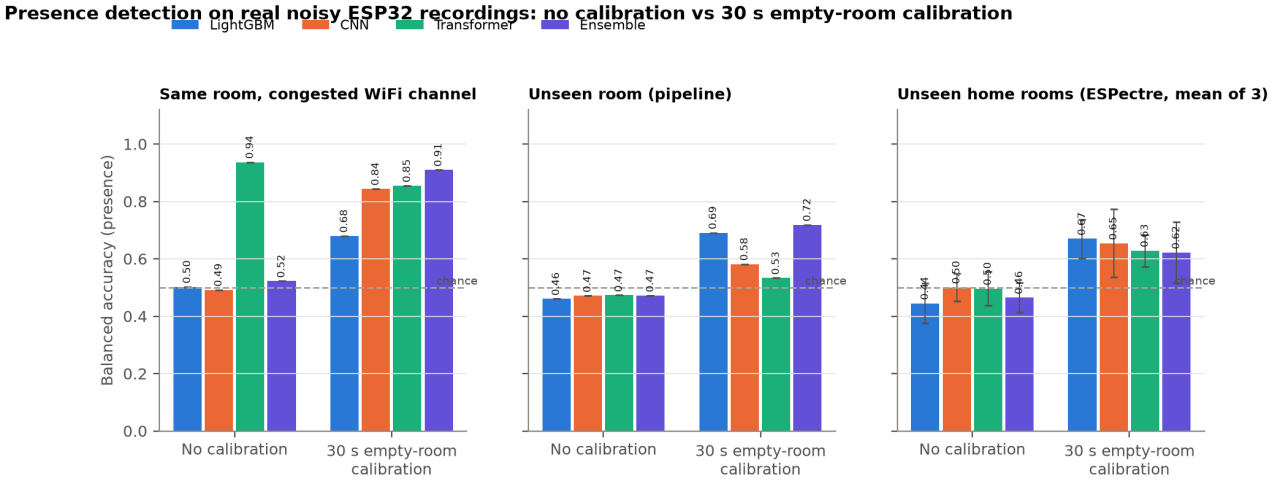



Figure 1: presence balanced accuracy, raw versus calibrated.

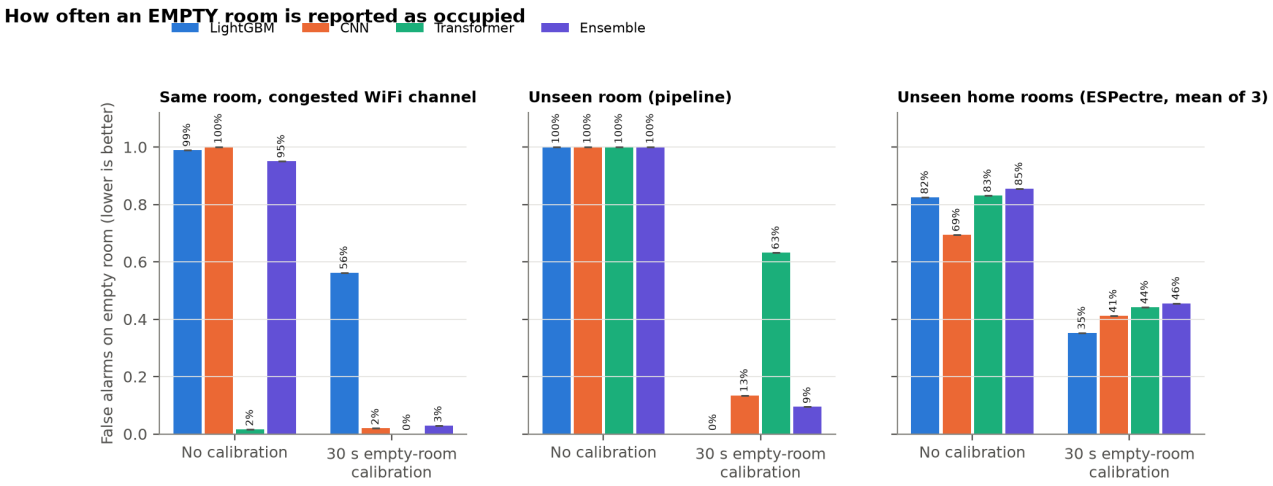
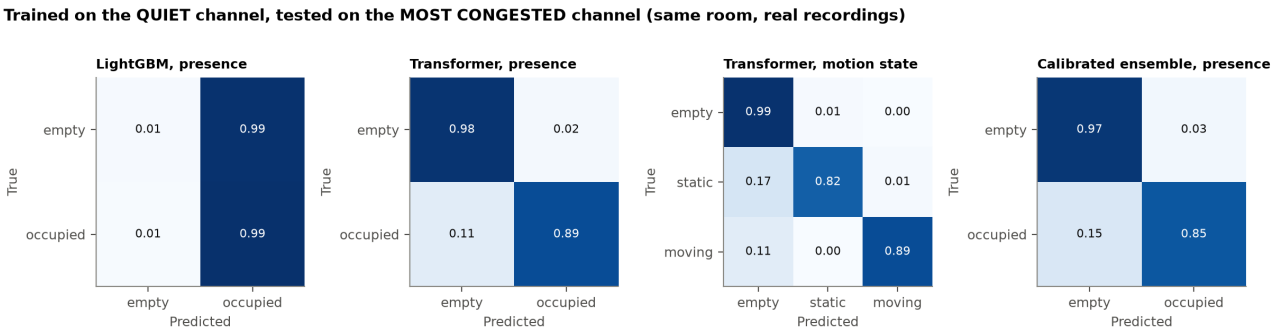
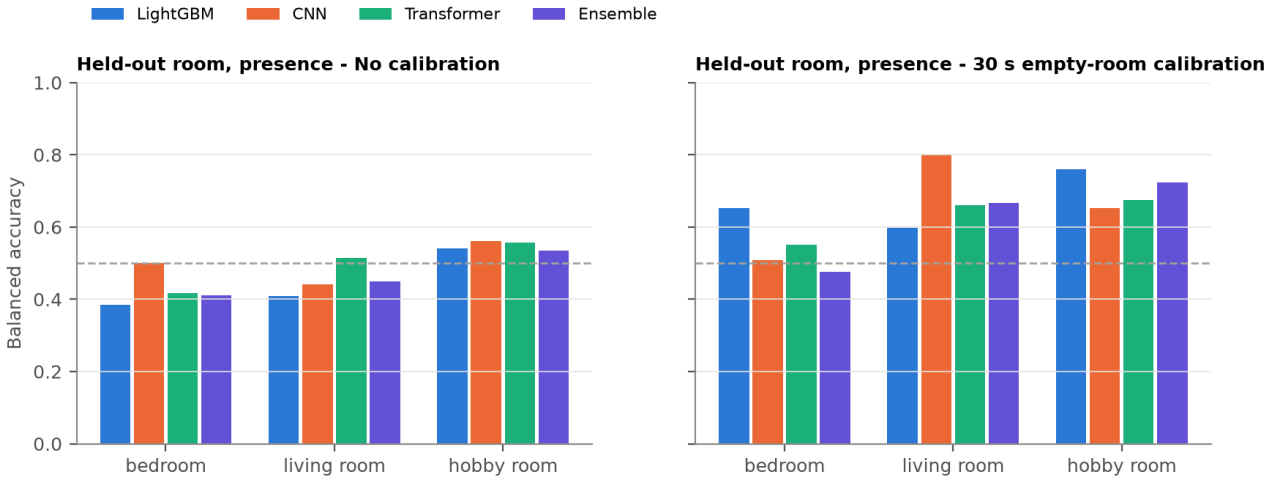


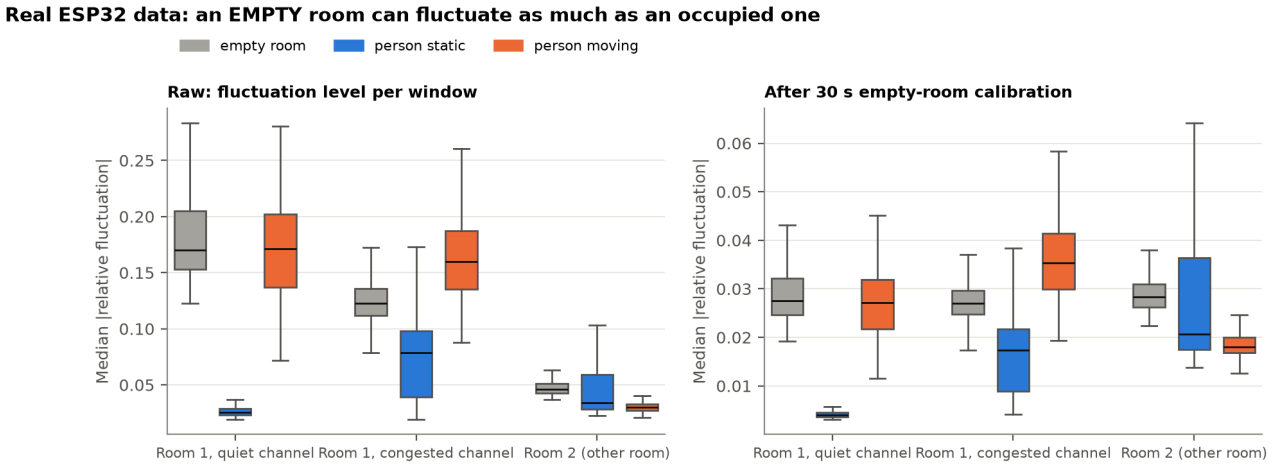

Figure 2: empty-room false-alarm rate.



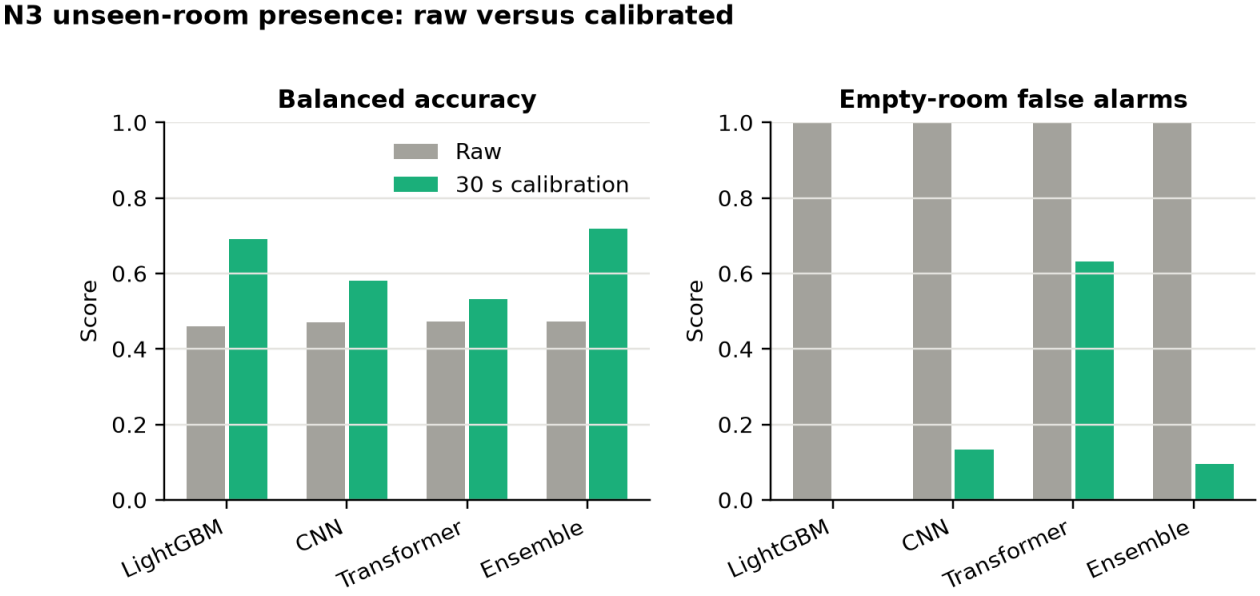

N2 confusion matrices on a measured congested channel.



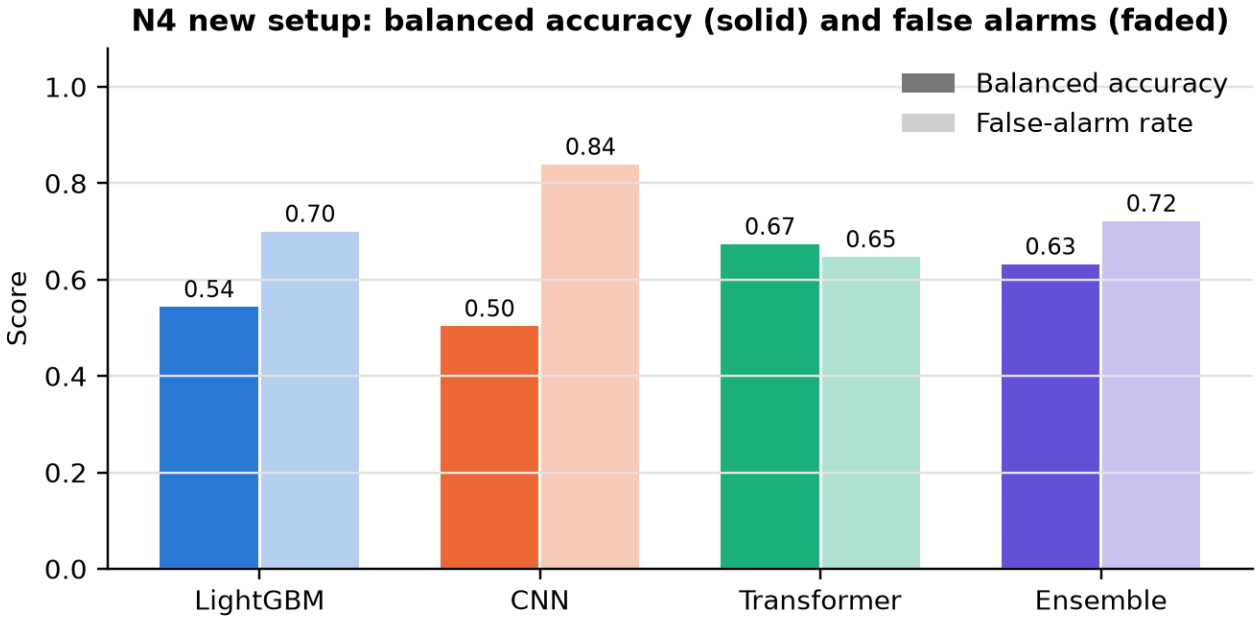

N1 held-out ESPectre-room comparison.



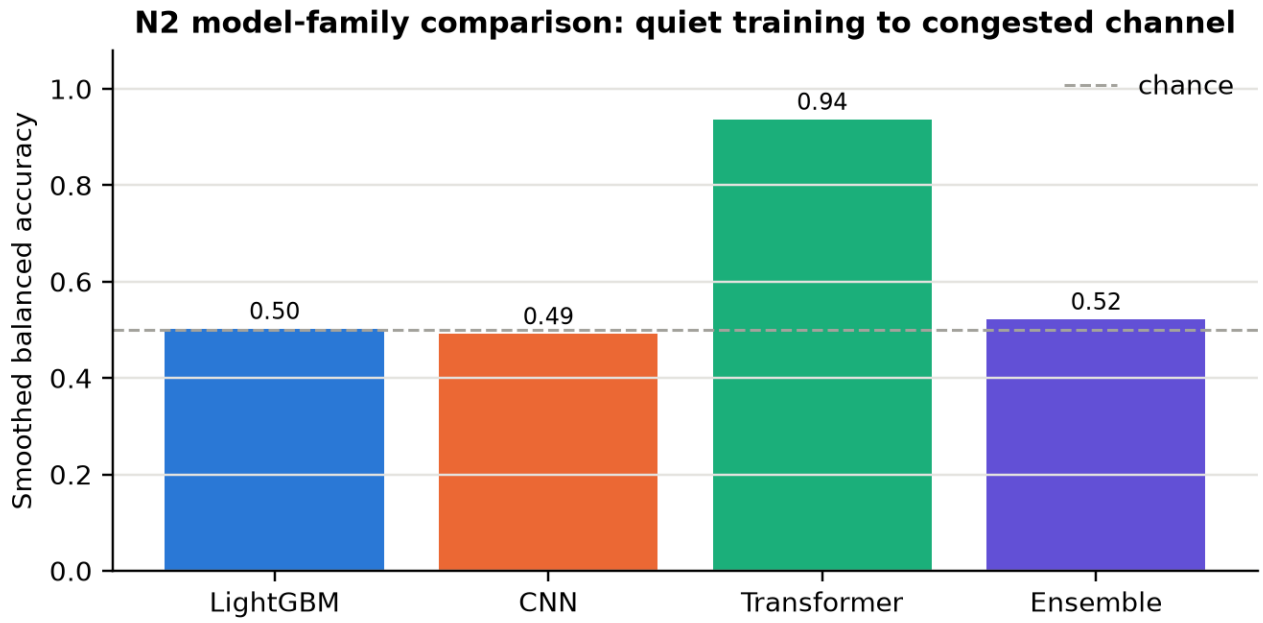

Empty versus occupied fluctuation distributions.



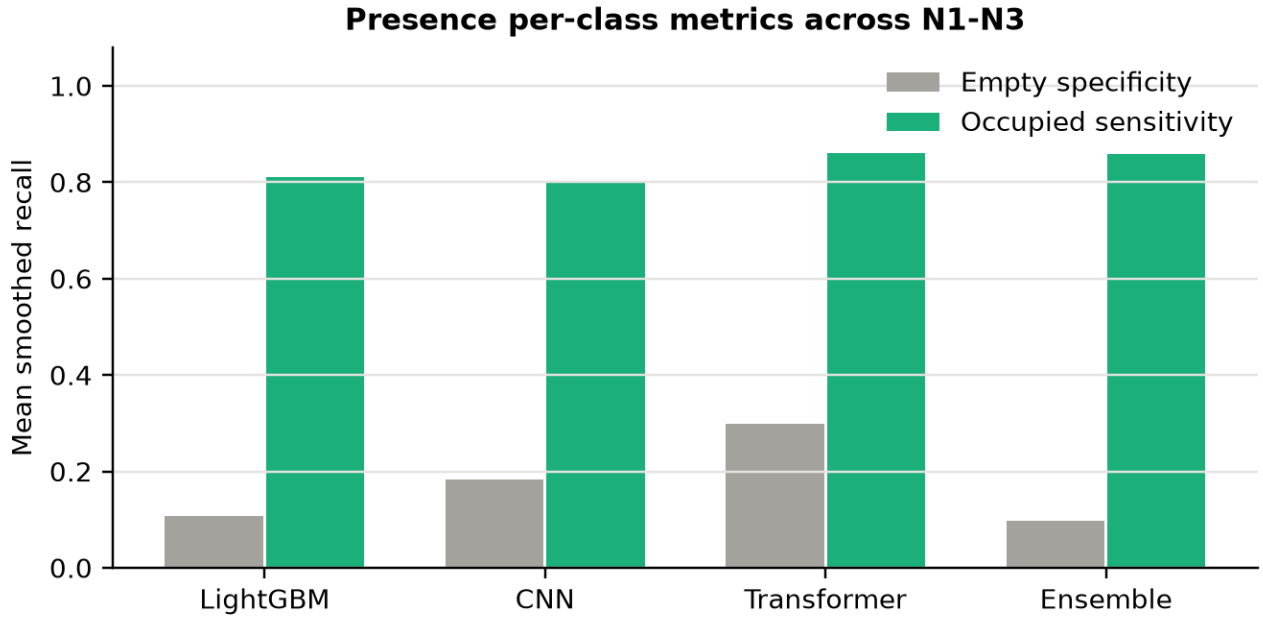

N3 unseen-room raw versus calibrated comparison.




N4 new-setup transfer comparison.




N2 model-family comparison.




Presence empty specificity and occupied sensitivity.

# 6. Reproduction and use

From the repository root, with Python 3.12 and requirements installed:

```
py -3.12 V3/src/show_transformer.py --task presence py -3.12 V3/src/show_transformer.py --task motion
```

These commands use cached pipeline windows and perform offline checkpoint loading plus one-window inference. They do not access an ESP32 and are not a live serial application.

Regenerate figures and registries:

```
py -3.12 V3/src/plots.py py -3.12 V3/src/dashboard_plots.py py -3.12 V3/src/build_registry.py
```

Run historical evaluations from V3/src only when intentionally retraining/evaluating:

```
py -3.12 experiments_noise.py N1 N2 N3 N4 py -3.12 experiments_noise.py N1c N2c N3c
```

Final full-pool model training, which can overwrite final filenames, is:

```
py -3.12 V3/src/train_final_v3.py --epochs 30
```

Model loading example:

```
import sys, torch sys.path.insert(0, 'V3/src') import deep state =
torch.load('V3/models/presence_tf.pt', map_location='cpu') model = deep.make_model('tf', n_classes=2)
model.load_state_dict(state)
```

For live capture, use the firmware and capture documentation under datasets/esp32_csi_pipeline/. The raw receiver schema must be checked first; complete live serial-to-model inference and Raspberry Pi latency have not been validated.

## Software

• Python 3.12 was used for current Windows validation.

• Root requirements include numpy, pandas, scipy, scikit-learn, lightgbm, matplotlib, reportlab, and torch.

• Capture requires pyserial>=3.5 and ESP-IDF only for flashing.

# 7. Limitations and next actions

• The CNN ignores the gap mask even though preprocessing produces it.

• Missing bins are interpolated, so the gap mask is required for interpretation.

• Calibrated paths need explicit DataFrame/NumPy index-alignment tests after filtering.

• Overlapping windows make random window splits invalid for generalization.

• Historical results do not include event-level smoothing delay or confidence intervals.

• Phase, complex CSI, and proposed feature groups have not been empirically validated.

• Live receiver format compatibility and Raspberry Pi real-time latency remain unvalidated.

## Recommended order

| Priority | Action | Reason |
|---|---|---|
| 1 | Test raw CSI_DATA schema, timestamp wrap, malformed packets, and gap masks | Prevents invalid live input |
| 2 | Assert index alignment in calibrated experiment paths | Protects new ablations from silent misalignment |
| 3 | Run grouped feature ablations A-G | Identify features that generalize beyond training rooms |
| 4 | Run controlled V4 HomeOccupancy fine-tuning | Test temporal home-domain adaptation separately |
| 5 | Validate live hardware and measure latency | Required before deployment claims |

## Source documents

V3_HANDOVER.md, QUICKSTART.md, MODEL_AND_RESULTS_INDEX.md, NEXT_STEPS.md, V3_REPORT.md, PROTOCOL.md, FEATURE_INVENTORY.md, LIMITATIONS.md, and protocol_feature_audit.md are under V3/docs/. The registry and cleanup records are under V3/manifests/ and V3/results/reports/.

End of handover